# AI Compiler Performance Engineering

Auto-Loops Books — ch06

# Contents

**1 Data Residency and On-Chip Memory Design** **1**

1.1 Lifetime planning . . . . . 1

1.2 GPU residency . . . . . 3

1.3 CPU residency . . . . . 5

1.4 NPU residency . . . . . 6

1.5 YiRage memory pass . . . . . 8

1.6 Key Takeaways . . . . . 9

1.7 Conclusion . . . . . 10

**Bibliography** **11**

**Index** **13**

# Chapter 1

# Data Residency and On-Chip Memory Design

Every decode bottleneck in this book reduces to one question: which bytes get to stay on chip, and for how long. Parts I–II established that batch-1 decode is bandwidth- and orchestration-bound, not FLOP-bound; this chapter turns that diagnosis into a design discipline—planning the lifetime and residency of every tensor against the concrete on-chip memory each target actually has (NVIDIA, 2022; AMD, 2024a; Dao *et al.*, 2022). The numbers from Chapter 1 are the motivation: dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 and OLMoE-1B/7B averages 9,305, and each unfused boundary is a tensor that left the chip and came back (Vellaisamy *et al.*, 2026). Removing host overhead alone bought 1.259× on a Qwen-2.5-7B decode cell, but it could not fix illegal HBM traffic—only residency design can (Chen, 2026; NVIDIA, 2024). The mistake to avoid is treating on-chip memory as an optimization to apply after the kernel works. On-chip residency is the kernel's contract with the hardware, and it must be planned before codegen, per target, because the same tensor that fits in a GPU's shared memory overflows an NPU tile's SRAM (SemiAnalysis, 2024; AMD, 2024b).

## 1.1 Lifetime planning

The first discipline is **lifetime** planning: for every intermediate tensor, determine when it is produced, when it is last consumed, and therefore where it should live for that span (Goodfellow *et al.*, 2016). A tensor whose producer and consumer

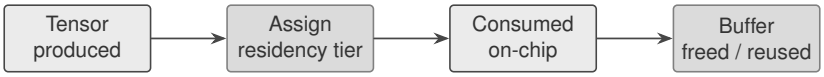



Figure 1.1: Lifetime planning assigns each tensor a residency tier for the span between its production and last use, then reuses the buffer.

are adjacent in a fused region never needs to touch device memory at all; a tensor consumed many operators later, or by a different thread block, has a longer lifetime and may be forced to a slower tier. Getting this wrong is the single most common cause of "mysterious" decode regressions, because a tensor with an unnecessarily long lifetime silently occupies fast memory that another tensor needed, or spills to HBM and reintroduces the bandwidth tax the fusion was meant to remove (Vellaisamy *et al.*, 2026; Chen, 2026).

Decode makes lifetime planning unusually tractable and unusually important. Tractable, because at batch-1 the per-token working set is small and regular: a query vector, the weight tiles for the current layer, and the KV cache slice for the current head (Kwon *et al.*, 2023). Important, because that working set is read in full every single token, so any byte that lives one tier slower than it could pays its penalty thousands of times per sequence (Davies *et al.*, 2025). The **residency** decision for the KV cache is the canonical example: it is too large to stay in the fastest tiers, so the design question becomes how much of it to stream and how to overlap that streaming with compute, not whether to keep it resident.

The KV cache deserves its own lifetime analysis because it breaks the simple producer-consumer pattern. Unlike an activation that is produced and consumed within a token, the KV cache is written once and then read on every subsequent token for the rest of the sequence, so its lifetime spans the entire generation. That long lifetime is why it cannot live in the fastest tiers and why decode is fundamentally a streaming problem: each token must read back the growing KV history, and the residency design question is not whether to keep it on chip but how to overlap its streaming with the per-token compute so the load latency is hidden (Kwon *et al.*, 2023; Dao *et al.*, 2023). A residency plan that ignores this and treats the KV slice like a short-lived activation will either run out of fast memory or stall waiting for the load.

Double buffering is the lifetime technique that makes streaming work, and it is worth naming because it appears on every hardware class. The idea is to give a streamed tensor two physical buffers so the engine can load tile $n{+}1$ into one buffer while compute consumes tile $n$ from the other, then swap. This doubles the buffer's footprint but overlaps transfer with compute, turning a sequence of load-then-compute stalls into a pipeline (SemiAnalysis, 2024; Dao *et al.*, 2022). The

lifetime planner must account for the doubled footprint when it checks capacity, which is the first place a naive plan and a real one diverge—a fused region that fits single-buffered may overflow once double buffering is added for the streamed operands.

A useful way to think about lifetime planning is as a static memory allocation problem solved at compile time rather than by a runtime allocator. If the compiler knows every tensor's lifetime, it can assign a small pool of physical buffers and reuse each one across non-overlapping lifetimes—exactly the static allocation and copy elimination that compilers like Glow perform on their low-level IR, and that a megakernel must perform to keep its working set bounded (Rotem *et al.*, 2018; Wu *et al.*, 2024). The payoff is twofold: the allocator never appears in the per-token hot path, and the peak **residency** footprint is known before the kernel runs, so it can be checked against the target's on-chip capacity instead of discovered by a runtime out-of-memory error.

There is a subtlety worth flagging: lifetimes are not fixed by the math, they are shaped by the schedule. The same dataflow graph can be scheduled so that a tensor's producer and consumer are adjacent—giving it a short lifetime and on-chip residency—or so that unrelated work is interleaved between them, stretching its lifetime and forcing it to a slower tier (Wu *et al.*, 2024; Goodfellow *et al.*, 2016). This is why residency planning and scheduling cannot be separated: the schedule that minimizes launches may lengthen a critical tensor's lifetime past what fast memory can hold, and the schedule that keeps everything resident may serialize work that could have overlapped. The planner therefore co-optimizes the two, choosing an execution order that keeps the highest-reuse tensors' lifetimes short enough to stay resident while still exposing enough overlap to hide streaming latency (Vellaisamy *et al.*, 2026; Chen, 2026).

The chapters that follow apply this discipline per hardware class, because the tiers themselves differ. What does not differ is the planning method: identify lifetimes, rank tensors by reuse-per-byte, and place the highest-reuse tensors in the fastest tier that can hold them for their lifetime, spilling only what genuinely cannot fit (Goodfellow *et al.*, 2016; Dao *et al.*, 2022).

## 1.2 GPU residency

On a GPU the fast tiers are concrete and small, and the design must respect their exact sizes. On an NVIDIA H100 (Hopper) SM, the **register** file is 65,536 32-bit registers—256 KB per SM—with a hard ceiling of 255 registers per thread; **shared** memory is configurable up to 228 KB per SM, of which 227 KB is addressable by a

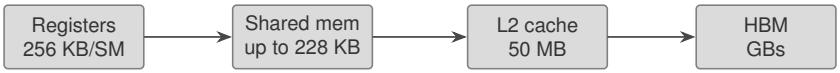



Figure 1.2: H100 on-chip residency tiers per SM: registers and shared memory are the fast, capacity-limited tiers a decode kernel must budget against.

single thread block after CUDA reserves 1 KB; and the combined L1-plus-**shared** capacity is 256 KB per SM, backed by a 50 MB GPU-wide L2 cache before HBM (NVIDIA, 2022). These are not trivia: they are the constraints a decode kernel's residency plan is solved against.

The **register**/occupancy trade is the first one a decode kernel hits. Because resident threads share the 65,536-register file, a kernel that uses $R$ registers per thread caps resident threads near $65536/R$; at 128 registers per thread an SM holds at most 512 threads (NVIDIA, 2022). A heavily fused decode kernel that keeps many partial sums in registers therefore trades occupancy for residency, and the right balance is workload-specific—at batch-1 there is little parallelism to hide latency, so keeping the working set in registers often beats chasing occupancy, up to the point where a register spill to local memory undoes the win.

**Shared** memory is where the canonical decode fusion lives. FlashAttention keeps the query tile, the relevant key/value tiles, and the running softmax statistics in **shared** memory and registers, computing attention without ever materializing the full score matrix to HBM (Dao *et al.*, 2022). The residency arithmetic is direct: at hidden size 4096 in BF16 each activation vector is 8 KB, so a fused decoder layer that keeps a handful of intermediates on chip avoids tens of kilobytes of HBM traffic per token—traffic that, multiplied across 32 layers and every generated token, dominates batch-1 latency (Vellaisamy *et al.*, 2026; Chen, 2026). Hopper adds mechanisms that make residency cheaper to exploit: TMA performs asynchronous bulk copies into **shared** memory that overlap with compute, and distributed shared memory lets thread blocks in a cluster read each other's shared memory directly rather than through L2 (SemiAnalysis, 2024; NVIDIA, 2022). The design lever is to size tiles so the hot working set fits in the 228 KB budget and to use the async copy path so streaming the KV slice overlaps the matmul rather than stalling it.

The bank structure of **shared** memory is a second constraint the residency plan must respect. H100 shared memory is divided into 32 banks, each 4 bytes wide, and consecutive 4-byte words map to consecutive banks; a warp that accesses 32 addresses falling in distinct banks completes in one transaction, while accesses that collide on a bank serialize into multiple transactions (NVIDIA, 2022). A residency plan that places a tile in shared memory but lays it out so threads stride across the same bank turns a fast on-chip access into a serialized one, which at

batch-1 decode—where the kernel runs thousands of times per sequence—is a measurable bytes/token penalty. The layout of a resident tensor is therefore part of the residency decision, not an afterthought: tile shapes and padding are chosen so the hot access pattern is conflict-free.

A worked register budget makes the occupancy trade concrete. Suppose a fused decode kernel keeps a query vector, several accumulator registers for the running softmax, and a weight fragment live in registers, totaling 96 registers per thread. With the 65,536-register file that allows roughly $65536/96 \approx 682$ resident threads per SM, comfortably above one full thread block; push the kernel to 200 registers per thread to fuse more work and resident threads fall to about 327, below a 512-thread block, so occupancy drops (NVIDIA, 2022). At batch-1 that occupancy loss is often acceptable because there is no batch parallelism to hide latency anyway, but only up to the spill threshold: once a thread needs more than 255 registers the compiler spills to local memory, which is HBM-backed, and the residency win is lost (Vellaisamy *et al.*, 2026; Chen, 2026). The design rule is to fuse aggressively in registers until just before the spill, then stop.

The L2 cache is the last on-chip line of defense. At 50 MB it can hold reused weight tiles or KV lines across thread blocks, and Hopper exposes L2 residency controls so a kernel can bias frequently reused data to stay cached (NVIDIA, 2022). But L2 is shared GPU-wide and far slower than shared memory, so relying on it is a fallback, not a plan—the residency design still aims to keep the per-token hot set in registers and **shared** memory.

## 1.3 CPU residency

On a CPU the residency tiers are the **cache** hierarchy, and the design discipline shifts from explicit placement to cache-aware blocking. Unlike GPU shared memory, which the programmer manages directly, CPU L1/L2/L3 caches are hardware-managed, so residency is controlled indirectly by the access pattern: tile the computation so the working set of each tile fits in a target **cache** level and is reused before eviction (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018). For decode this means blocking the per-token GEMV and attention so the weight tile and KV slice currently in use stay in L1 or L2 across their reuse, rather than streaming the whole layer through the **cache** once and evicting it.

The second CPU-specific concern is **NUMA**. On multi-socket or multi-die servers, memory is partitioned across nodes, and a thread accessing memory on a remote **NUMA** node pays a latency and bandwidth penalty (Goodfellow *et al.*, 2016). For a memory-bound decode workload that re-reads the full weight

set every token, a **NUMA**-oblivious placement—weights allocated on one node, decode threads scheduled on another—can silently halve effective bandwidth. The residency plan on a CPU therefore includes thread and memory affinity: pin decode threads to the **NUMA** node that owns the weights they read, and replicate read-only weights per node when the bandwidth saving exceeds the memory cost (Davies *et al.*, 2025).

The bandwidth arithmetic explains why this matters so much for decode on a CPU. At batch-1 the model re-reads its entire weight set from memory every token, so the achievable token rate is bounded by weight bytes divided by effective memory bandwidth—a memory-bound regime where a 7B model in BF16 must move roughly 14 GB per token from wherever the weights live (Kwon *et al.*, 2023; Chen, 2026). If a **NUMA**-oblivious schedule forces half those reads across the inter-node link at a fraction of local bandwidth, the token rate drops proportionally, and no amount of compute optimization recovers it. This is the CPU analogue of the GPU's launch storm: the bottleneck is bytes moved through the wrong tier, and the fix is a residency and placement decision, not a faster inner loop.

Hardware prefetchers complicate the **cache** residency picture in a way GPU shared memory does not. A CPU prefetcher will speculatively pull data into **cache** based on detected access patterns, which helps a streaming weight read but can also evict a small, hot tensor the kernel wanted to keep resident—so the blocking strategy must consider not just what fits in a **cache** level but what the prefetcher will displace (Goodfellow *et al.*, 2016). The practical consequence is that CPU residency tuning is more empirical than GPU shared-memory tuning: the plan is validated by measuring **cache** miss rates at the decode operating point rather than by a static capacity check alone.

The multi-hardware contrast with the GPU is instructive. A GPU kernel decides residency explicitly and statically through **shared**-memory allocation; a CPU kernel decides it implicitly through blocking and affinity, trusting the hardware **cache** to keep the blocked working set resident. The same fused decoder layer therefore wants a different schedule on each: the GPU schedule maximizes **shared**-memory reuse and async copy overlap, while the CPU schedule maximizes **cache** locality and respects **NUMA** boundaries—identical math, different residency mechanism, different bytes/token outcome (Vellaisamy *et al.*, 2026; Chen, 2026).

## 1.4 NPU residency

On a spatial **NPU** the residency model is the most explicit of all, and it must be decided entirely at compile time. On the AMD XDNA2 / AIE-ML fabric, each

compute tile has 64 KB of L1 local data memory—eight 8 KB banks in hardware, presented to the programmer as four interleaved 16 KB banks—with a dedicated DMA engine, and each column adds a 512 KB L2 **SRAM** memory tile shared by its compute tiles, with shim tiles bridging to off-chip DDR (AMD, 2024a,b). There is no hardware cache that opportunistically keeps data resident; what lives in **SRAM** is exactly what the compiler's DMA schedule placed there, and nothing else.

This makes **NPU** residency design both unforgiving and powerful. Unforgiving, because a tensor that does not fit the 64 KB tile **SRAM** must be split across tiles or staged through the 512 KB memory tile, and the DMA chain to do so is part of the program, not a runtime fallback (AMD, 2024b). Powerful, because once the residency plan fits, there is no cache-miss variance and no allocator—the per-token cost is deterministic. The canonical decode fusion on this fabric keeps the attention tiles resident in L1/L2 **SRAM** and streams only inputs and final outputs through DDR, mapping the query-key-softmax-value chain as a single spatially tiled sweep so that intermediate scores never leave on-chip **SRAM** (AMD, 2024a; Dao *et al.*, 2022). Tiling parameters are chosen to maximize L1 reuse subject to the buffer constraint that the resident tiles fit the tile **SRAM**.

Double buffering on the **NPU** is both more explicit and more essential than on the GPU. Because there is no cache to absorb a late load, a compute tile that waits for its next operand simply idles, so the DMA schedule must ping-pong between two buffers in the 64 KB tile **SRAM**—loading the next tile via the tile's DMA engine while the vector unit consumes the current one (AMD, 2024a,b). This halves the usable tile capacity for the double-buffered operand, which is precisely the kind of constraint the residency plan must enforce up front: on a 64 KB tile, a double-buffered weight tile plus a double-buffered activation tile plus the accumulators must all fit, and if they do not, the tiling parameters shrink until they do.

The spatial structure adds a residency dimension the GPU and CPU lack: data can move between neighboring tiles without going off chip. On the XDNA2 fabric, compute tiles in an array can read each other's local memory directly and pass data through cascade connections, so a large operand can be distributed across several tiles' **SRAM** and consumed as a spatial pipeline rather than streamed from DDR (AMD, 2024a,b). A residency plan that exploits this maps a fused attention sweep across a row or column of tiles, keeping the intermediate scores entirely in on-chip **SRAM** and sending only the inputs and final outputs through DDR—the spatial analogue of a GPU megakernel, with the same goal of minimizing off-chip bytes per token (Dao *et al.*, 2022; Chen, 2026).

Table 1.1: On-chip residency tiers by hardware class; the fast tiers a decode kernel must budget against differ in size and in who manages them (NVIDIA, 2022; AMD, 2024a).

| Class | Fast on-chip tier | Managed by |
|---|---|---|
| GPU (H100 SM) | 228 KB shared, 256 KB registers | Programmer + HW L2 |
| CPU | L1/L2/L3 cache | Hardware (blocking-tuned) |
| NPU (XDNA2 tile) | 64 KB L1, 512 KB L2 SRAM | Compiler DMA (explicit) |

The contrast across the three classes is now sharp. The GPU offers a large, programmer-managed **shared** memory plus a hardware L2 fallback; the CPU offers hardware-managed caches tuned by blocking; the **NPU** offers small, fully explicit **SRAM** with compiler-scheduled DMA and no fallback. A residency plan that is optimal on one is illegal or wasteful on another, which is exactly why on-chip memory design cannot be a single portable template (SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

## 1.5 YiRage memory pass

The book's reference compiler turns this discipline into a checkable pass. **YiRage** carries a **memory**-residency pass that encodes the per-class residency contract—GPU shared-memory and register budgets, CPU cache-blocking and NUMA constraints, NPU tile-SRAM capacity—as IR metadata, then verifies a candidate schedule against the target's actual on-chip **memory** before codegen (Chen and YiRage Contributors, 2026). A fusion that would exceed the 228 KB shared-memory budget on an H100, or overflow a 64 KB XDNA tile, is rejected at compile time rather than discovered as a spill in production telemetry (NVIDIA, 2022; AMD, 2024a).

This is the mechanism behind the residency claims made throughout the book. When earlier chapters said a schedule must "keep the layer on chip," the **YiRage memory** pass is what makes that statement enforceable: it computes the peak resident footprint of a fused region from the lifetime plan, compares it to the target tier capacity, and either admits the fusion or splits it until it fits (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Because the check is static, it composes with the megakernel search of Chapter 20—the superoptimizer proposes a fusion, and the **memory** pass certifies that its residency is legal on the target before the kernel is ever emitted (Wu *et al.*, 2024; Mirage Project contributors, 2025).

Concretely, the pass operates on the lifetime plan rather than on the source

program, which is what lets it reason about peak footprint correctly. It walks the fused region, tracks which buffers are simultaneously live at each point, accounts for double-buffered operands at their doubled size, and takes the maximum across the region as the peak resident footprint (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). That maximum is compared against the target tier capacity—228 KB on an H100 SM, 64 KB on an XDNA tile—and if it exceeds the budget the pass has two legal responses: split the region so each piece fits, or demote the lowest-reuse tensor to a slower tier and accept its bandwidth cost (NVIDIA, 2022; AMD, 2024a). Both are deterministic compile-time decisions, so the same model compiled for two SKUs gets two different but provably-legal residency plans.

The pass also closes a gap that pure search leaves open. A superoptimizer that proposes a megakernel is optimizing for performance and may propose a fusion that is faster in principle but illegal on a given tier; without a residency check, that fusion ships and spills in production (Wu *et al.*, 2024; Mirage Project contributors, 2025). By making residency a verified precondition, the **YiRage memory** pass lets the search be aggressive—propose any fusion—while guaranteeing that whatever is emitted respects the target's on-chip **memory**, which is the separation of concerns that makes searched megakernels safe to deploy across a heterogeneous fleet (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

The result is that residency stops being a property a kernel author hopes for and becomes one the compiler guarantees. For a serving team that is the difference between a decode kernel that is fast on the SKU it was tuned on and one that is provably within budget on every SKU in the fleet (Davies *et al.*, 2025; Chen, 2026).

## 1.6 Key Takeaways

- **Plan tensor lifetimes before codegen.** Determine each tensor's production-to-last-use span and place it in the fastest tier that holds it for that lifetime; static allocation keeps the allocator out of the per-token path and makes the peak footprint checkable (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018). A too-long lifetime silently spills (Vellaisamy *et al.*, 2026).

- **Budget against real GPU tier sizes.** An H100 SM has 256 KB of registers (255/thread max) and up to 228 KB of shared memory; fused decode kernels trade occupancy for residency and must fit the hot set in these budgets (NVIDIA, 2022). FlashAttention keeps tiles in shared memory to avoid HBM traffic (Dao *et al.*, 2022).

- **CPU residency is blocking plus NUMA.** Caches are hardware-managed,

so residency is controlled by cache-blocked tiling and by pinning decode threads to the NUMA node owning their weights (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). A NUMA-oblivious placement can halve effective bandwidth on a memory-bound decode loop (Chen, 2026).

- **NPU residency is explicit and compile-time.** An XDNA2 tile has 64 KB L1 SRAM and a 512 KB column L2, with compiler-scheduled DMA and no cache fallback, so what is resident is exactly what the schedule placed (AMD, 2024a,b). Tiles are sized to fit the SRAM buffer constraint (SemiAnalysis, 2024).
- **Make residency a checkable compiler pass.** The YiRage memory pass encodes per-class budgets as IR metadata and rejects fusions that exceed on-chip capacity before codegen, composing with megakernel search (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Residency becomes a guarantee the compiler proves, not a hope the kernel author holds (Mirage Project contributors, 2025).

## 1.7 Conclusion

On-chip residency is the concrete form every decode optimization in this book eventually takes: plan each tensor's lifetime, rank by reuse-per-byte, and place the hot working set in the fastest tier that can hold it on the target you actually ship (Goodfellow *et al.*, 2016; NVIDIA, 2022). The tiers differ sharply—programmer-managed shared memory and registers on a GPU, hardware caches plus NUMA on a CPU, explicit DMA-scheduled SRAM on an NPU—so the same fused layer wants a different residency plan per class, and a green build on one says nothing about residency on another (AMD, 2024a; Chen, 2026). The YiRage memory pass makes the plan enforceable by checking peak footprint against tier capacity before codegen, so a fusion that would spill is split or demoted rather than shipped (Chen and YiRage Contributors, 2026). With residency now a budgeted, checkable property, Chapter **??** turns to the next MegaKernel technique built on it: static role assignment and schedule-free kernels that fix each thread block's job at compile time so the resident working set never moves (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

# Bibliography


AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 7, 8, 9, 10

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 7, 10

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 9, 10

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 8, 9, 10

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 2

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 7, 9

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 2, 6, 9, 10

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 3, 5, 6, 8, 9, 10

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 2, 6

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. `https://github.com/mirage-project/mirage`. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; 1.2–6.7× latency reduction. 8, 9, 10

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 1, 4, 5, 8, 9, 10

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1

Rotem, N., Fix, J., Abdulrasool, S., Catron, G., Deng, S., Dzhabarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*. Two-phase strongly-typed IR (high-level graph + low-level address-only); lowering to linear-algebra primitives; profile-guided quantization; AOT bundles. 3, 5, 9

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 4, 8, 10

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 9, 10

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. $\mu$Graph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9×. 3, 8, 9, 10

# Index

Data residency, 1

On-chip memory, 1

Shared memory, 1